

Python Football Engine

Football Analysis, Statistical Modelling
and Prediction Engine

A Python-based football analytics project developed from first principles, progressing from raw historical match data through feature engineering, statistical modelling, machine learning, prediction and betting-market analysis.

Benjamin Fielding

Mathematics with Finance

Newcastle University

Project Journal

Contents

1	Project Introduction	1
1.1	Project Overview	1
1.2	Initial Project Structure	1
2	Phase 1: Data Preparation	2
2.1	Loading the Raw Dataset	2
2.2	Selecting Relevant Variables	2
2.3	Date Conversion	3
2.4	Missing Data	3
2.5	Reusable Data Setup Function	3
3	Phase 2: Team-Level Match Data	4
3.1	The Home/Away Problem	4
3.2	Selecting Matches for a Team	5
3.3	Home and Away Indicator	5
3.4	Opponent Variable	5
3.5	Goals Scored and Conceded	6
3.6	Shots	7
3.7	Shots on Target	8
3.8	Corners	8
3.9	Clean Team View	8
3.10	The team_view Function	9
4	Phase 3: Historical Match Retrieval	11
4.1	Motivation	11
4.2	Desired Historical Match Function	11
4.3	Default Date Behaviour	12
4.4	User-Supplied Dates	12
4.5	Filtering Previous Matches	12
4.6	Sorting the Historical Matches	13
4.7	Selecting the Previous n Matches	13
4.8	Input Validation	14

4.9	The previous Function	15
4.10	Testing the Function	16
4.11	Early-Season Behaviour	16
5	Current Engine Architecture	17
6	Progress Summary	18
6.1	Phase 1: Data Preparation	18
6.2	Phase 2: Team-Level Data	18
6.3	Phase 3: Historical Match Retrieval	19
7	Next Stage: Feature Engineering	19

1. Project Introduction

1.1 Project Overview

The aim of this project is to construct a football analysis and prediction engine in Python.

The project will progressively transform raw historical football match data into meaningful statistical information, predictive features and eventually probabilistic match forecasts.

The engine will be developed gradually rather than immediately applying a machine learning algorithm to the raw dataset.

The broad structure of the project is

Data → Cleaning → Exploration → Feature Engineering → Statistics

→ Regression → Machine Learning → Predictions → Odds / Value Analysis → Evaluation.

This structure allows each stage of the modelling process to be understood before moving to the next.

The initial development of the engine uses only one Premier League season, 2022/23. Restricting the analysis to a single season reduces unnecessary complexity while the underlying data-processing architecture is developed.

Once the engine functions correctly on one season, additional seasons can be introduced.

1.2 Initial Project Structure

The project is currently separated into several Python files:

```
FootballEngine/  
|  
|-- main.py  
|-- functions.py  
|-- test_page.py  
|-- E0_223.csv
```

Each file has a different purpose.

main.py acts as the main walkthrough of the project. It records the development of the engine in a readable order and demonstrates how each part of the system was constructed.

functions.py contains reusable functions that form the underlying engine.

test_page.py provides a separate environment in which functions can be tested without cluttering the main project walkthrough.

E0_223.csv contains the raw Premier League match data for the 2022/23 season.

2. Phase 1: Data Preparation

2.1 Loading the Raw Dataset

The project begins by importing pandas and loading the raw Premier League CSV file.

```
1 import pandas as pd
2
3 data = pd.read_csv("E0_223.csv")
```

Each row of the dataframe represents one Premier League fixture.

The original dataset contains a large number of columns describing match results, match statistics and betting-market information.

Before constructing any models, it is important to understand which variables are currently required.

2.2 Selecting Relevant Variables

The first version of the engine focuses on basic match-performance statistics.

The variables retained are:

- **Date** — date of the fixture,
- **HomeTeam** — home club,
- **AwayTeam** — away club,
- **FTHG** — full-time home goals,
- **FTAG** — full-time away goals,
- **FTR** — full-time result,
- **HS** — home-team shots,
- **AS** — away-team shots,
- **HST** — home shots on target,
- **AST** — away shots on target,
- **HC** — home corners,
- **AC** — away corners.

Rather than deleting the large number of unwanted columns, a new dataframe is created containing only the required information.

```
1 football_data = data[
2     [
3         "Date",
4         "HomeTeam",
5         "AwayTeam",
6         "FTHG",
```

```

7         "FTAG",
8         "FTR",
9         "HS",
10        "AS",
11        "HST",
12        "AST",
13        "HC",
14        "AC"
15    ]
16 ].copy()

```

Using `.copy()` ensures that the new dataframe is independent of the original dataframe.

2.3 Date Conversion

When the CSV file is initially loaded, the `Date` column is stored as text.

However, the engine will eventually need to perform chronological operations such as:

- finding matches before a particular fixture,
- calculating recent form,
- constructing chronological training and testing datasets,
- preventing future data from being used in historical predictions.

The date column is therefore converted into pandas datetime format.

```

1 football_data["Date"] = pd.to_datetime(
2     football_data["Date"],
3     format="%d/%m/%Y"
4 )

```

The dataset is then sorted chronologically.

```

1 football_data = football_data.sort_values("Date")

```

2.4 Missing Data

The selected variables were inspected for missing observations using

```

1 football_data.isna().sum()

```

No missing values were found in the selected variables for the 2022/23 dataset.

This means that no additional missing-value treatment is currently required.

2.5 Reusable Data Setup Function

The entire preparation process was then converted into the first reusable engine function.

```

1 def data_setup(data):
2
3     football_data = data[
4         [
5             "Date",
6             "HomeTeam",
7             "AwayTeam",
8             "FTHG",
9             "FTAG",
10            "FTR",
11            "HS",
12            "AS",
13            "HST",
14            "AST",
15            "HC",
16            "AC"
17        ]
18    ].copy()
19
20    football_data["Date"] = pd.to_datetime(
21        football_data["Date"],
22        format="%d/%m/%Y"
23    )
24
25    football_data = football_data.sort_values("Date")
26
27    return football_data

```

The raw dataset can therefore be prepared using

```

1 data = pd.read_csv("E0_223.csv")
2
3 football_data = data_setup(data)

```

The first stage of the engine can therefore be represented as

Raw CSV $\longrightarrow$ data_setup() $\longrightarrow$ Clean Football Data.

3. Phase 2: Team-Level Match Data

3.1 The Home/Away Problem

The original football dataset is match-centred rather than team-centred.

For example, suppose Arsenal play Chelsea.

If Arsenal are at home, Arsenal's goals are stored in

FTHG.

However, if Arsenal are away, Arsenal's goals are stored in

FTAG.

The same issue occurs for shots, shots on target and corners.

Therefore, analysing a particular team becomes unnecessarily difficult if the raw home and away columns are used directly.

The objective of Phase 2 is to transform the raw match representation into a team-centred representation.

3.2 Selecting Matches for a Team

Suppose the selected team is Arsenal.

A match should be included if Arsenal appear either as the home team or as the away team.

The corresponding pandas condition is

```
1 team_games = football_data[
2     (football_data["HomeTeam"] == team) |
3     (football_data["AwayTeam"] == team)
4 ].copy()
```

The operator `|` represents an element-wise logical OR operation across the two pandas Boolean Series.

3.3 Home and Away Indicator

A new variable is created to identify whether the selected team was playing at home or away.

Initially, every fixture is classified as a home game.

```
1 team_games["Home_Away"] = "Home"
```

Rows where the selected team appears in the `AwayTeam` column are then changed.

```
1 team_games.loc[
2     team_games["AwayTeam"] == team,
3     "Home_Away"
4 ] = "Away"
```

The `loc` command can be interpreted as:

For the rows satisfying this condition, modify this named column.

This becomes a useful general pandas pattern throughout the project.

3.4 Opponent Variable

The next variable identifies the opponent.

The away team is initially used as the default opponent.

```
1 team_games["Opponent"] = team_games["AwayTeam"]
```

However, if the selected team is playing away, the opponent must instead be the home team.

```

1 team_games.loc[
2     team_games["Home_Away"] == "Away",
3     "Opponent"
4 ] = team_games["HomeTeam"]

```

For example,

Arsenal vs Chelsea

produces

Opponent = Chelsea,

while

Liverpool vs Arsenal

produces

Opponent = Liverpool.

3.5 Goals Scored and Conceded

The raw dataset records goals from the perspective of the home and away teams.

The engine instead requires the quantities

Scored

and

Conceded

from the perspective of the selected club.

For a home fixture,

Scored = FTHG,

and

Conceded = FTAG.

For an away fixture,

Scored = FTAG,

and

$$\text{Conceded} = \text{FTHG}.$$

This is implemented using

```
1 team_games["scored"] = team_games["FTHG"]
2
3 team_games.loc[
4     team_games["Home_Away"] == "Away",
5     "scored"
6 ] = team_games["FTAG"]
```

and

```
1 team_games["conceded"] = team_games["FTAG"]
2
3 team_games.loc[
4     team_games["Home_Away"] == "Away",
5     "conceded"
6 ] = team_games["FTHG"]
```

3.6 Shots

The same transformation is applied to shots.

For a home fixture,

$$\text{Shots For} = HS,$$

$$\text{Shots Against} = AS.$$

For an away fixture,

$$\text{Shots For} = AS,$$

$$\text{Shots Against} = HS.$$

The corresponding implementation is

```
1 team_games["shots_for"] = team_games["HS"]
2
3 team_games.loc[
4     team_games["Home_Away"] == "Away",
5     "shots_for"
6 ] = team_games["AS"]
7
8
9 team_games["shots_against"] = team_games["AS"]
10
11 team_games.loc[
```

```

12     team_games["Home_Away"] == "Away",
13     "shots_against"
14 ] = team_games["HS"]

```

3.7 Shots on Target

The same approach gives

```

1 team_games["sot_for"] = team_games["HST"]
2
3 team_games.loc[
4     team_games["Home_Away"] == "Away",
5     "sot_for"
6 ] = team_games["AST"]
7
8
9 team_games["sot_against"] = team_games["AST"]
10
11 team_games.loc[
12     team_games["Home_Away"] == "Away",
13     "sot_against"
14 ] = team_games["HST"]

```

3.8 Corners

Finally, corners are transformed into team-relative quantities.

```

1 team_games["corners_for"] = team_games["HC"]
2
3 team_games.loc[
4     team_games["Home_Away"] == "Away",
5     "corners_for"
6 ] = team_games["AC"]
7
8
9 team_games["corners_against"] = team_games["AC"]
10
11 team_games.loc[
12     team_games["Home_Away"] == "Away",
13     "corners_against"
14 ] = team_games["HC"]

```

3.9 Clean Team View

Once the new variables have been constructed, a cleaner dataframe is created.

```

1 team_view = team_games[
2     [
3         "Date",
4         "Opponent",
5         "Home_Away",
6         "scored",
7         "conceded",
8         "shots_for",

```

```

9         "shots_against",
10        "sot_for",
11        "sot_against",
12        "corners_for",
13        "corners_against"
14    ]
15 ].copy()

```

This gives a representation of each fixture entirely from the perspective of the selected team.

3.10 The team_view Function

The full transformation is packaged into the reusable function `team_view()`.

```

1 def team_view(football_data, team_name):
2
3     team = team_name
4
5     team_games = football_data[
6         (football_data["HomeTeam"] == team) |
7         (football_data["AwayTeam"] == team)
8     ].copy()
9
10    team_games["Home_Away"] = "Home"
11
12    team_games.loc[
13        team_games["AwayTeam"] == team,
14        "Home_Away"
15    ] = "Away"
16
17    team_games["Opponent"] = team_games["AwayTeam"]
18
19    team_games.loc[
20        team_games["Home_Away"] == "Away",
21        "Opponent"
22    ] = team_games["HomeTeam"]
23
24    team_games["scored"] = team_games["FTHG"]
25
26    team_games.loc[
27        team_games["Home_Away"] == "Away",
28        "scored"
29    ] = team_games["FTAG"]
30
31    team_games["conceded"] = team_games["FTAG"]
32
33    team_games.loc[
34        team_games["Home_Away"] == "Away",
35        "conceded"
36    ] = team_games["FTHG"]
37
38    team_games["shots_for"] = team_games["HS"]
39
40    team_games.loc[
41        team_games["Home_Away"] == "Away",
42        "shots_for"
43    ] = team_games["AS"]
44

```

```

45     team_games["shots_against"] = team_games["AS"]
46
47     team_games.loc[
48         team_games["Home_Away"] == "Away",
49         "shots_against"
50     ] = team_games["HS"]
51
52     team_games["sot_for"] = team_games["HST"]
53
54     team_games.loc[
55         team_games["Home_Away"] == "Away",
56         "sot_for"
57     ] = team_games["AST"]
58
59     team_games["sot_against"] = team_games["AST"]
60
61     team_games.loc[
62         team_games["Home_Away"] == "Away",
63         "sot_against"
64     ] = team_games["HST"]
65
66     team_games["corners_for"] = team_games["HC"]
67
68     team_games.loc[
69         team_games["Home_Away"] == "Away",
70         "corners_for"
71     ] = team_games["AC"]
72
73     team_games["corners_against"] = team_games["AC"]
74
75     team_games.loc[
76         team_games["Home_Away"] == "Away",
77         "corners_against"
78     ] = team_games["HC"]
79
80     team_view = team_games[
81         [
82             "Date",
83             "Opponent",
84             "Home_Away",
85             "scored",
86             "conceded",
87             "shots_for",
88             "shots_against",
89             "sot_for",
90             "sot_against",
91             "corners_for",
92             "corners_against"
93         ]
94     ].copy()
95
96     return team_view

```

The same function can now be applied to any club contained within the dataset.

For example,

```

1  arsenal_stats = team_view(
2      football_data,
3      "Arsenal"

```

```
4 )  
5  
6 chelsea_stats = team_view(  
7     football_data,  
8     "Chelsea"  
9 )
```

Phase 2 therefore transforms the engine from

Match-Centred Data

into

Team-Centred Data.

4. Phase 3: Historical Match Retrieval

4.1 Motivation

The eventual objective of the engine is to make predictions about football fixtures.

Any statistic used to predict a historical fixture must only use information that existed before that fixture occurred.

Suppose the engine wishes to predict a match taking place on

1 January 2023.

The engine cannot use matches from February, March or any later date when constructing the team's current form.

Doing so would introduce

Look-Ahead Bias.

Avoiding look-ahead bias is therefore a fundamental requirement of the feature-engineering process.

4.2 Desired Historical Match Function

A new function is required which accepts

- a team,
- a number of matches n ,
- an optional cutoff date,

and returns the previous n matches played by that team before the specified date.

Conceptually,

Team History $\rightarrow$ Date $<$ Cutoff $\rightarrow$ Newest First $\rightarrow$ First n Games.

4.3 Default Date Behaviour

The function uses

```
1 date=None
```

as the default argument.

If no date is supplied, the engine should return the latest available matches in the dataset.

Therefore,

```
1 if date is None:
2     date = data["Date"].max() + pd.Timedelta(days=1)
```

is used.

The additional day is required because matches are later selected using the strict inequality

Date $<$ date.

If the maximum date itself were used, the final fixture would be excluded.

4.4 User-Supplied Dates

If a date is supplied manually, it must be converted into pandas datetime format before it can be compared with the `Date` column.

A flexible day-first conversion is used:

```
1 date = pd.to_datetime(
2     date,
3     dayfirst=True
4 )
```

This allows the function to interpret dates using the standard British day-month-year convention.

4.5 Filtering Previous Matches

The team-level match history is first generated using

```
1 team_stats = team_view(
2     data,
3     team_name
4 )
```

Only matches occurring before the cutoff date are retained:

```

1 previous_games = team_stats[
2     team_stats["Date"] < date
3 ]

```

This pandas Boolean filter applies the condition to every row in the dataframe.

4.6 Sorting the Historical Matches

The original dataframe is stored chronologically from oldest to newest.

However, the historical-match function requires the most recent fixture to appear first.

Therefore,

```

1 previous_games = previous_games.sort_values(
2     "Date",
3     ascending=False
4 )

```

reverses the chronological ordering.

4.7 Selecting the Previous n Matches

After sorting the dataframe, the previous n matches are obtained using positional indexing:

```

1 previous_games = previous_games.iloc[0:n].copy()

```

This introduced the distinction between `loc` and `iloc`.

The useful mental rule is

`loc` = labels or conditions

while

`iloc` = integer positions.

For example,

```

1 df.loc[
2     df["Home_Away"] == "Away",
3     "scored"
4 ]

```

uses a Boolean condition and a column label.

By contrast,

```

1 df.iloc[0:5]

```

selects rows based purely on their positions.

4.8 Input Validation

Basic input validation was added before processing the historical matches.

The first requirement is

$$n > 0.$$

This is implemented as

```
1 if n <= 0:
2     raise ValueError(
3         "n must be greater than 0"
4     )
```

The function also checks whether the requested team exists in the dataset.

Let

$$A = \{\text{team appears in HomeTeam}\}$$

and

$$B = \{\text{team appears in AwayTeam}\}.$$

A valid team satisfies

$$A \vee B.$$

Therefore, an invalid team satisfies

$$\neg(A \vee B).$$

By De Morgan's law,

$$\neg(A \vee B) = (\neg A) \wedge (\neg B).$$

The corresponding Python implementation is

```
1 if not (
2     (team_name in data["HomeTeam"].values) or
3     (team_name in data["AwayTeam"].values)
4 ):
5     raise ValueError(
6         "Invalid team name"
7     )
```

This also demonstrates the distinction between Python logical operators and pandas element-wise operators.

For individual Boolean values,

and, or

are used.

For pandas Boolean Series,

&, |

are generally used.

4.9 The previous Function

Combining the above steps produces the historical-match retrieval function.

```
1 def previous(n, data, team_name, date=None):
2
3     # Input validation
4     if n <= 0:
5         raise ValueError(
6             "n must be greater than 0"
7         )
8
9     if not (
10         (team_name in data["HomeTeam"].values) or
11         (team_name in data["AwayTeam"].values)
12     ):
13         raise ValueError(
14             "Invalid team name"
15         )
16
17     # Date handling
18     if date is None:
19         date = (
20             data["Date"].max()
21             + pd.Timedelta(days=1)
22         )
23
24     else:
25         date = pd.to_datetime(
26             date,
27             dayfirst=True
28         )
29
30     # Generate team history
31     team_stats = team_view(
32         data,
33         team_name
34     )
35
36     team_stats = team_stats.reset_index(
37         drop=True
38     )
39
40     # Select matches before cutoff
41     previous_games = team_stats[
42         team_stats["Date"] < date
```

```

43     ]
44
45     # Most recent match first
46     previous_games = previous_games.sort_values(
47         "Date",
48         ascending=False
49     )
50
51     # Take first n observations
52     previous_games = previous_games.iloc[
53         0:n
54     ].copy()
55
56     return previous_games

```

4.10 Testing the Function

The function was tested on several teams and different historical dates.

For example,

```

1 test1 = previous(
2     5,
3     football_data,
4     "Arsenal"
5 )

```

returns Arsenal's latest five matches in the dataset.

A historical Chelsea example is

```

1 test2 = previous(
2     5,
3     football_data,
4     "Chelsea",
5     "01/01/2023"
6 )

```

which returns the five Chelsea fixtures immediately preceding 1 January 2023.

A larger historical window can be requested using

```

1 test3 = previous(
2     10,
3     football_data,
4     "Liverpool",
5     "01/10/2022"
6 )

```

4.11 Early-Season Behaviour

An important case occurs when the requested number of previous matches is greater than the number of fixtures that had actually been played.

For example,

```
1 test4 = previous(  
2     10,  
3     football_data,  
4     "Arsenal",  
5     "15/08/2022"  
6 )
```

may request ten previous fixtures even though Arsenal had only played two matches before this date.

Pandas handles this naturally.

The command

```
1 previous_games.iloc[0:n]
```

returns all available observations up to n .

Therefore, requesting ten fixtures when only two exist simply returns those two matches rather than producing an error.

This behaviour will be particularly important once historical pre-match features are generated automatically.

5. Current Engine Architecture

At the end of Phase 3, the engine has the following structure:

Raw Premier League CSV



data_setup()



Clean Match-Level Data



team_view()



Team-Centred Match History



previous()



Previous n Matches Before Any Specified Date

This represents the fundamental data infrastructure required before meaningful football features can be constructed.

6. Progress Summary

6.1 Phase 1: Data Preparation

- ✓ Load one Premier League season.
- ✓ Inspect the raw dataset.
- ✓ Select relevant football variables.
- ✓ Convert dates into pandas datetime objects.
- ✓ Sort fixtures chronologically.
- ✓ Check selected variables for missing values.
- ✓ Construct reusable `data_setup()` function.

6.2 Phase 2: Team-Level Data

- ✓ Select all fixtures involving a chosen team.
- ✓ Identify whether the team was home or away.
- ✓ Identify the opponent.
- ✓ Create goals scored and conceded.
- ✓ Create shots for and against.
- ✓ Create shots on target for and against.
- ✓ Create corners for and against.
- ✓ Construct clean team-centred dataframe.
- ✓ Construct reusable `team_view()` function.

6.3 Phase 3: Historical Match Retrieval

- ✓ Filter fixtures before a specified date.
- ✓ Support a default date.
- ✓ Convert user-supplied dates.
- ✓ Sort historical matches newest to oldest.
- ✓ Retrieve the previous n fixtures.
- ✓ Handle cases with fewer than n available matches.
- ✓ Validate positive values of n .
- ✓ Validate team names.
- ✓ Test the function across multiple teams and dates.
- ✓ Construct reusable `previous()` function.

Phase 3 Complete

7. Next Stage: Feature Engineering

The next phase of the project will begin transforming historical match data into quantitative pre-match features.

The function

`previous()`

provides the historical window required to calculate these quantities.

For a lookback window containing n matches, potential features include

$$\text{Average Goals Scored} = \frac{1}{n} \sum_{i=1}^n G_i^{\text{for}},$$

and

$$\text{Average Goals Conceded} = \frac{1}{n} \sum_{i=1}^n G_i^{\text{against}}.$$

Similar quantities can be calculated for

- shots for,
- shots against,
- shots on target for,

- shots on target against,
- corners for,
- corners against.

Additional features may later include

- wins,
- draws,
- losses,
- points accumulated,
- recent form,
- home-only performance,
- away-only performance,
- weighted recent performance.

The central modelling restriction will remain:

Every feature used to predict a fixture must be calculated only from
information available before that fixture occurred.

This will provide the foundation for the statistical and machine learning sections of the football engine.